UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
CENTRO UNIVERSITARIO DE OCCIDENTE DIVISIÓN DE CIENCIAS DE LA INGENIERÍA
INGENIERÍA EN CIENCIAS Y SISTEMAS
ESTRUCTURA DE DATOS
PROYECTO 1 - FASE 2
PRIMER SEMESTRE 2026
Andy Jefferson González Fuentes 202231338

MANUAL DE USUARIO

El sistema gestiona un catálogo de productos de supermercado usando cinco estructuras de datos implementadas desde cero en C++: lista enlazada no ordenada, lista enlazada ordenada, árbol AVL, árbol B y árbol B +. Todas las estructuras se mantienen sincronizadas mediante inserción atómica con **rollback**. Cómo instalar, compilar y usar la aplicación de consola de Gestión de Catálogo de Supermercado. El programa permite agregar, buscar, eliminar y listar productos usando cinco estructuras de datos implementadas desde cero.

Herramienta	Versión mínima	Propósito
g++ / clang++	C++17	Compilación del proyecto
CMake	3.16	Sistema de construcción
Graphviz	Cualquiera	Generar imagenes PNG de los arboles

Instalar en Ubuntu/Debian:

```
sudo apt update && sudo apt install build-essential cmake graphviz
```

Estructura del proyecto

Carpeta / Archivo	Contenido
src/	Implementaciones .cpp de todas las clases
frontend	Implementación de la vista con angular
include/	Headers .h de todas las clases
data/productos.csv	CSV con 1000 productos de prueba
data/productos demo.csv	CSV con 20 productos para visualizar arboles
resultados/	Archivos .dot, .png y errors.log generados al ejecutar
build/	Ejecutable generado por CMake
CMakeLists.txt	Configuración de compilación
README.md	Instrucciones rápidas de compilacion y ejecucion

Compilación

Desde la raíz del proyecto (donde esta CMakeLists.txt):

```
mkdir build
```

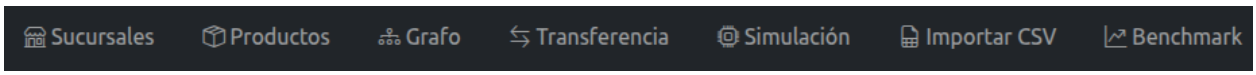
```
cd build
cmake ..
make
```

El ejecutable se genera en build/supermercado. Para recompilar después de cambios en archivos existentes:

```
cd build && make
```

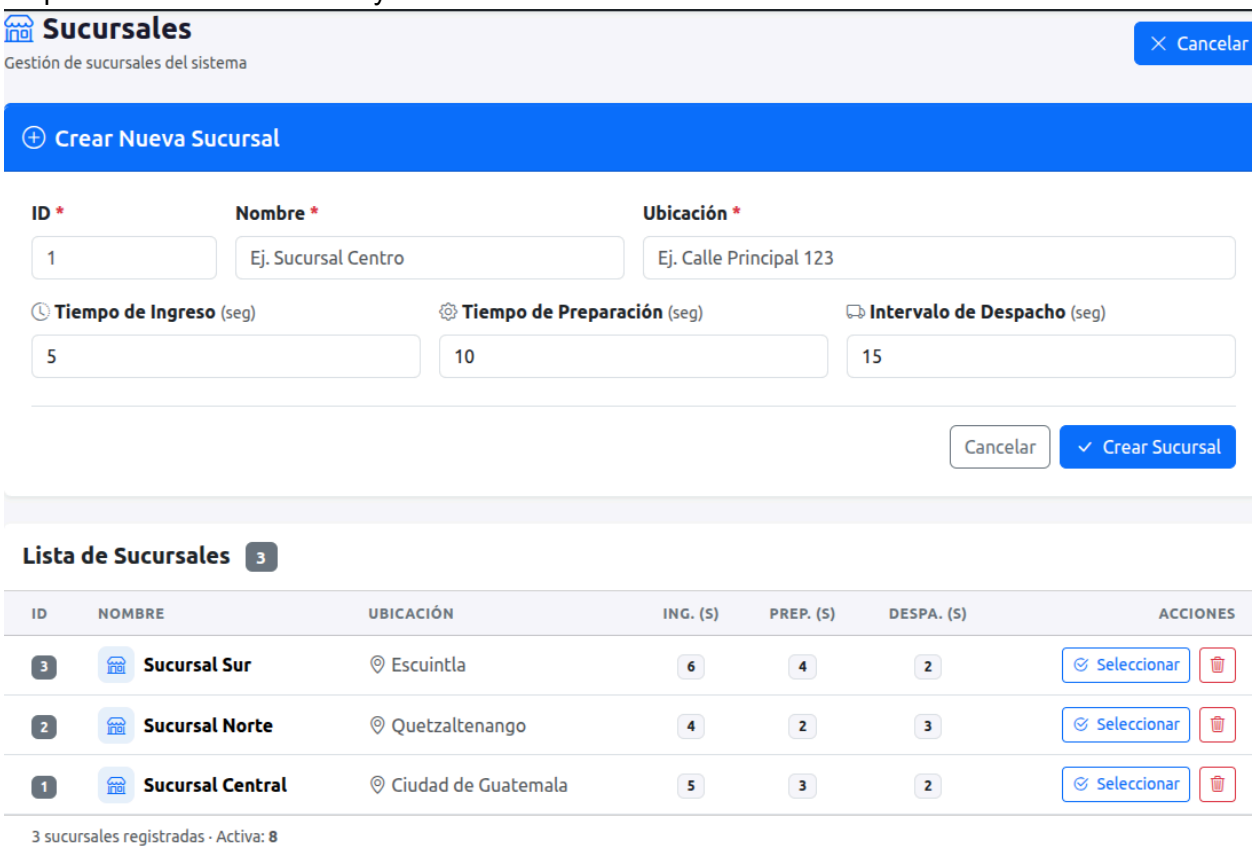
Menú principal

Al iniciar el programa se muestra el menú con las siguientes opciones:



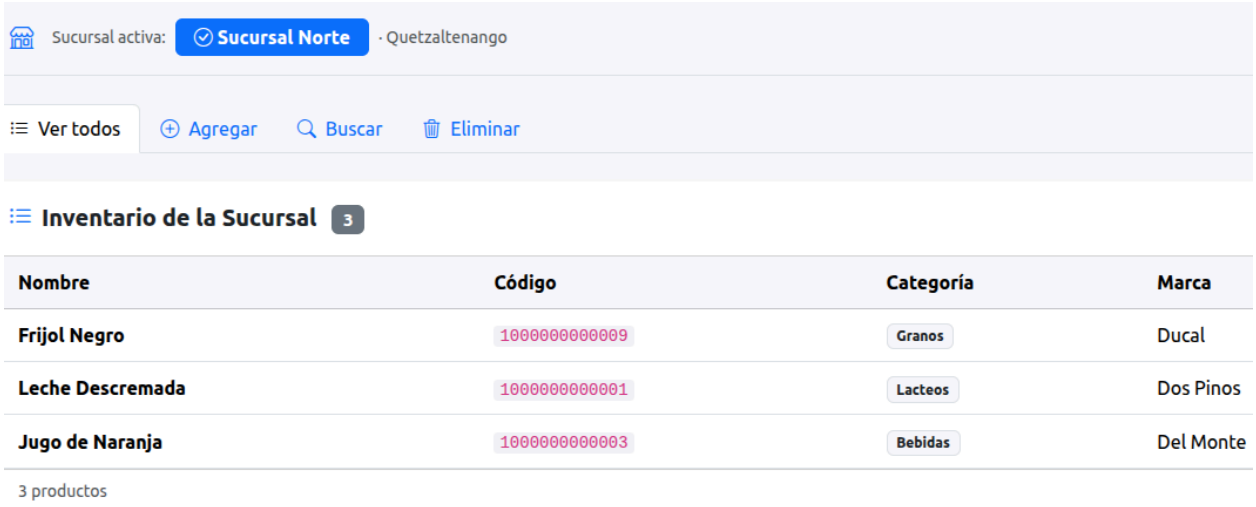
Sucursales:

Se pueden crear sucursales y listar las sucursales desde un archivo csv



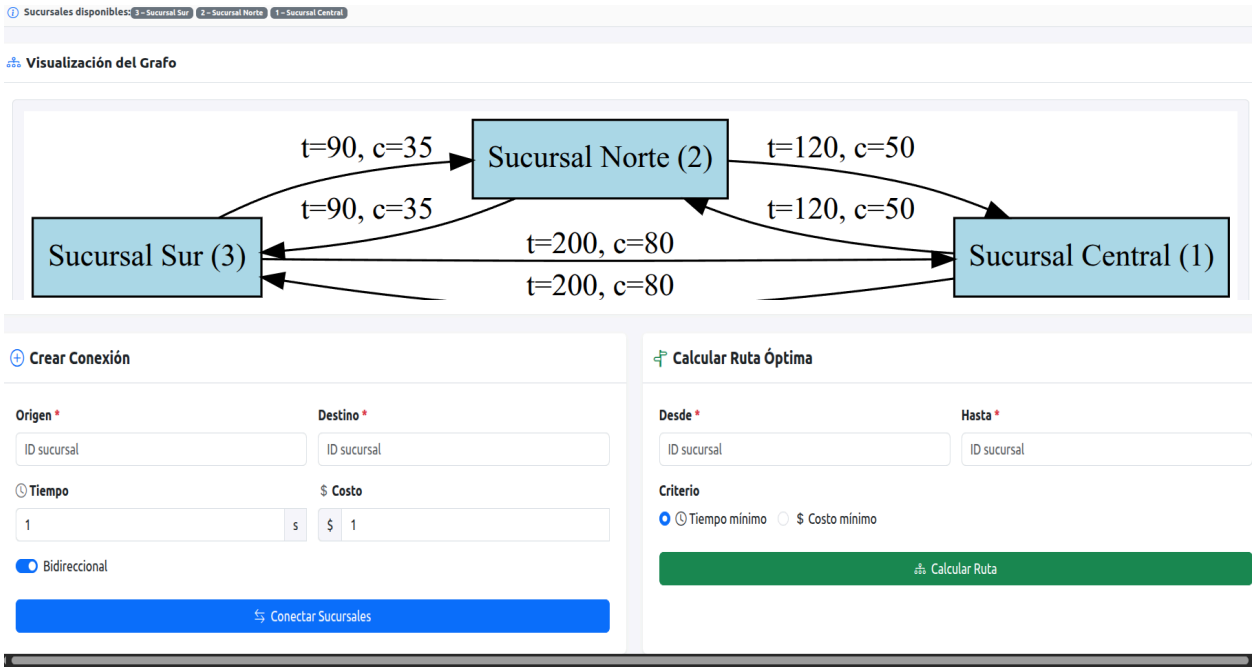
Productos de una sucursal:

Se activa una sucursal la cual contiene productos y sus respectivas interacciones.



Grafos de sucursales:

Contiene la visualización de las conexiones entre sucursales por tiempo y por dinero, permite la realización de conexiones y calcular la ruta óptima usando el algoritmo de dijkstra.



Transferencia:

Las transferencias son el traslado de productos de una sucursal a otra mandando un producto y la cantidad en stock. Esta se realiza calculando la ruta más corta, además respeta tiempos y estatus de los productos, mostrando en todo momento si un transacción está en las colas de ingreso, preparación y salida.

Agregar Transferencia

Sucursal Origen

Sucursal Central (ID: 1)

Sucursal Destino

Sucursal Sur (ID: 3)

Producto

Seleccionar...

Cantidad

1

Criterio

⏱️ Tiempo 💰 Costo

Etiqueta (opcional)

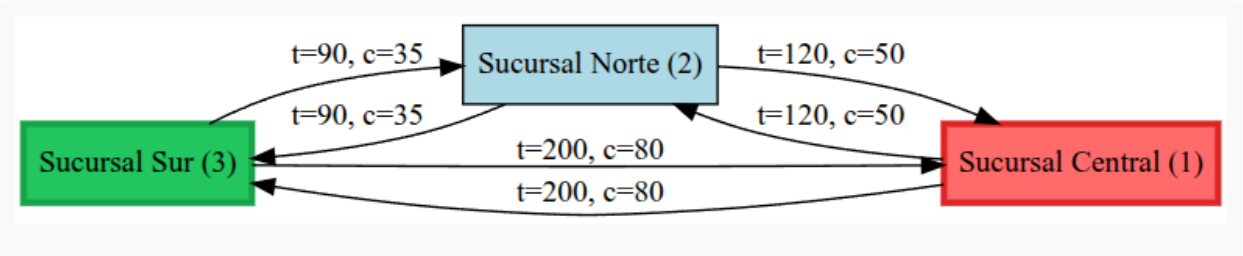
ej: Urgente

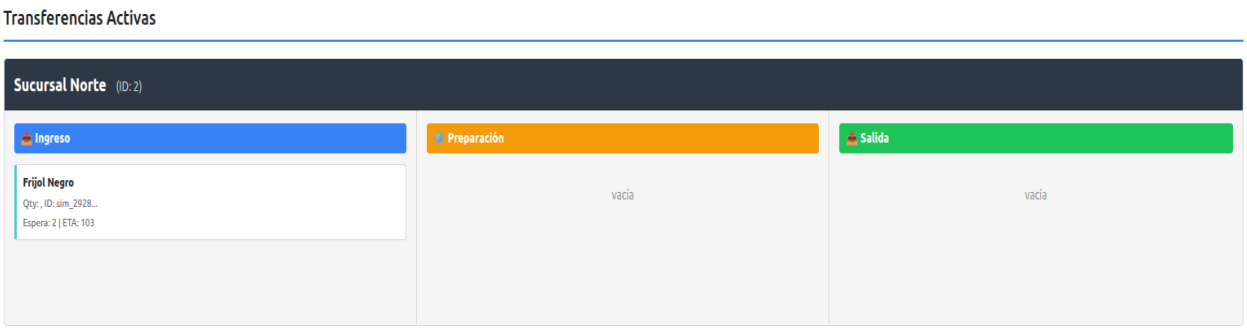
Ver Ruta | **Agregar al Batch**

Últimas Transferencias Completadas

Transfer ID	Código	Ruta	Tick	Estado
sim_907104_1...	1000000000002	Sucursal Central → Sucursal Sur	1151	✓ OK

Red de Sucursales



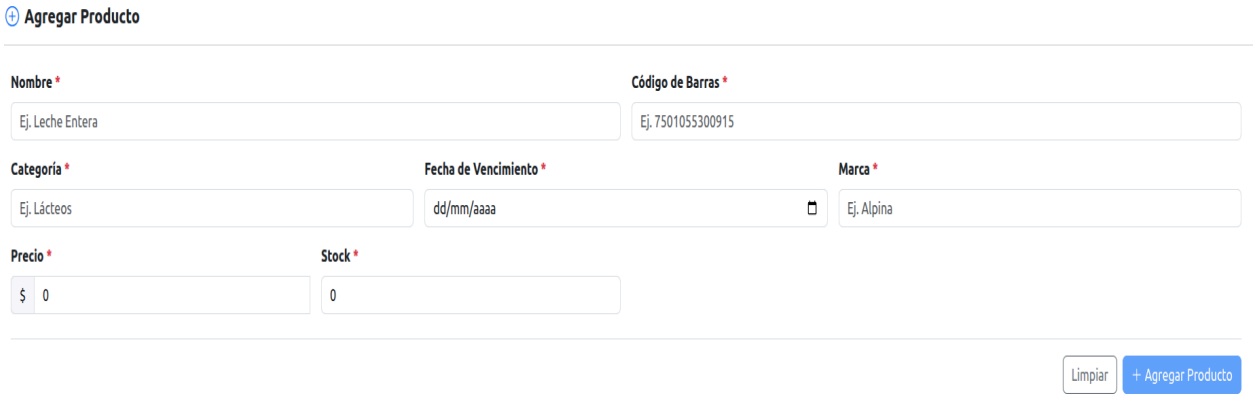


Operaciones detalladas dentro de sucursales.

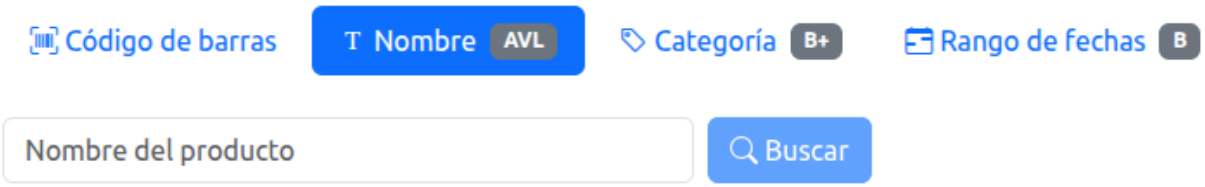
1. Agregar producto: El sistema solicita los siguientes campos uno por uno.

Campo	Formato	Ejemplo
Nombre	Texto libre	Leche Entera
Codigo de barra	Alfanumérico único	7501055300001
Categoría	Texto libre	Lácteos
Fecha caducidad	YYYY-MM-DD	2026-06-15
Marca	Texto libre	Lala
Precio	Número decimal > 0	18.50
Stock	Entero >= 0	100

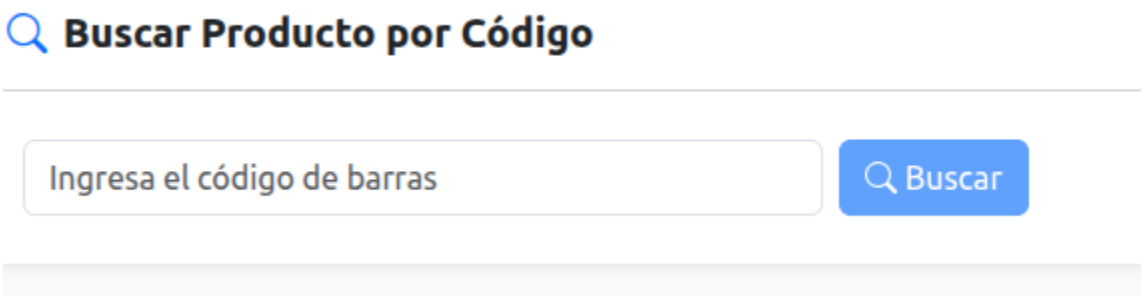
Si el código de barra ya existe en el catálogo, la inserción se rechaza y se registra en **errors.log**. Si algún campo es invalido, el producto no se inserta.



2. Buscar por nombre: Ingresa el nombre exacto del producto. El sistema realiza una búsqueda binaria en el árbol AVL y muestra todos los campos del producto si lo encuentra.



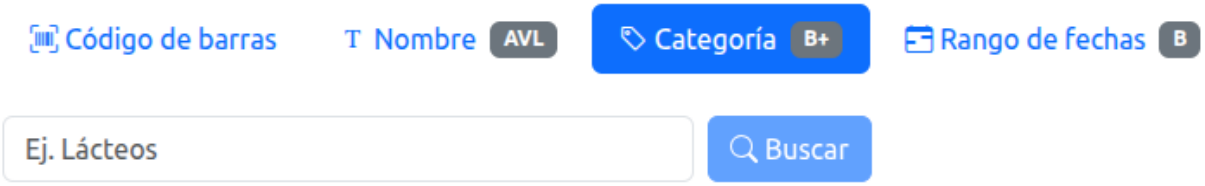
3. <u>Buscar por código de barra:</u> Ingresa el código de barra exacto. El sistema recorre la lista enlazada secuencialmente. Retorna el producto si existe o un mensaje de no encontrado.



4. <u>Buscar por categoría:</u> Ingresa el nombre de la categoría (ej. Lacteos, Bebidas). El sistema usa el árbol B+ para retornar todos los productos de esa categoría, mostrados en tabla de 3 columnas con paginación.

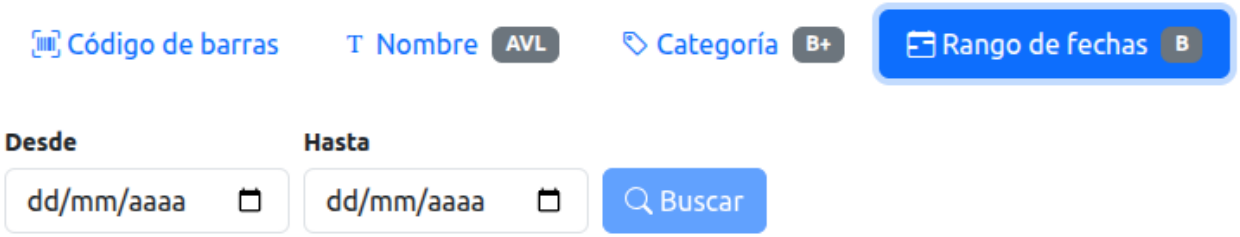
   Navegación en la tabla:

   S - siguiente página
   A - página anterior
   Q - volver al menú



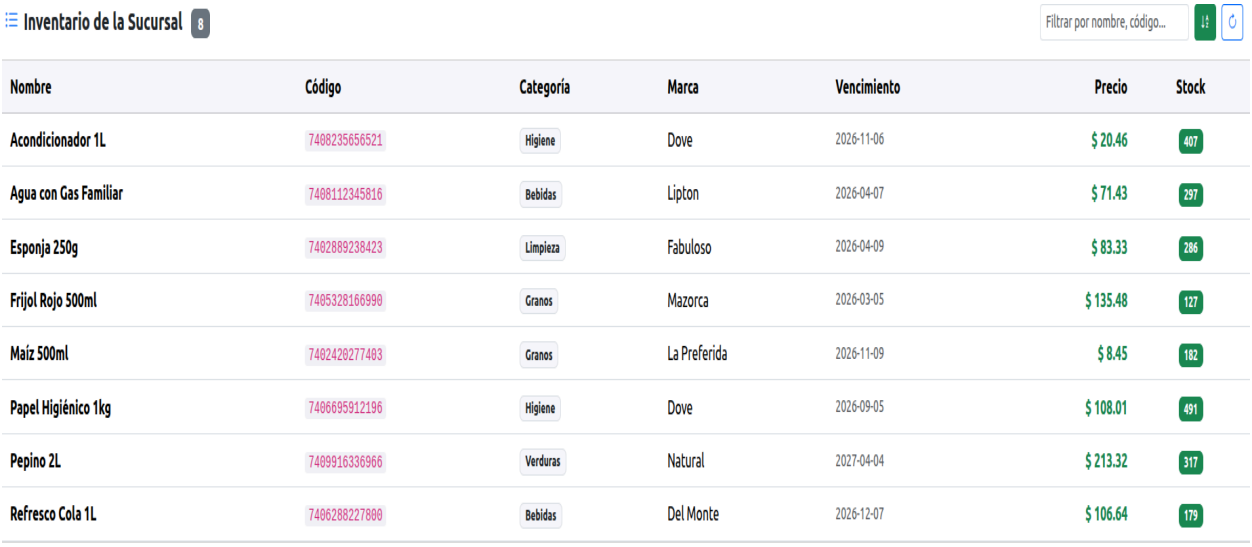
5. <u>Buscar por rango de fechas:</u> Ingresa una fecha de inicio y una fecha de fin en formato **YYYY-MM-DD**. El sistema usa el árbol B para retornar todos los productos cuya fecha de caducidad esté dentro del rango indicado.

   Ejemplo: fecha inicio 2026-01-01 y fecha fin 2026-06-30 retorna todos los productos que caducan en el primer semestre de 2026.



6. <u>Eliminar producto:</u> Ingresa el código de barra del producto a eliminar. El sistema lo elimina de las cinco estructuras de forma **sincronizada**. Si el código no existe, muestra un mensaje de error.

7. <u>Listar todos en orden alfabético:</u> Muestra todos los productos ordenados alfabéticamente por nombre en una tabla de 3 columnas. Cada producto muestra todos sus campos. La tabla está paginada con navegación S/A/Q.



Nombre	Código	Categoría	Marca	Vencimiento	Precio	Stock
Acondicionador 1L	7408235656521	Higiene	Dove	2026-11-06	$ 20.46	407
Agua con Gas Familiar	7408112345816	Bebidas	Lipton	2026-04-07	$ 71.43	297
Esponja 250g	7402889238423	Limpieza	Fabuloso	2026-04-09	$ 83.33	286
Frijol Rojo 500ml	7405328166990	Granos	Mazorca	2026-03-05	$ 135.48	127
Maíz 500ml	7402420277403	Granos	La Preferida	2026-11-09	$ 8.45	182
Papel Higiénico 1kg	7406695912196	Higiene	Dove	2026-09-05	$ 108.01	491
Pepino 2L	7409916336966	Verduras	Natural	2027-04-04	$ 213.32	317
Refresco Cola 1L	7406288227880	Bebidas	Del Monte	2026-12-07	$ 106.64	179

8. <u>Cargar CSV:</u> Ingresa la ruta al archivo CSV. El sistema valida cada linea y carga los productos validos. Los errores se registran en **resultados/errors.log** sin interrumpir la carga.

Formato esperado del CSV:

SUCURSALES

`"ID","Nombre","Ubicacion","tiempoIngreso","tiempoPreparacion","intervaloDespa"`

CONEXIONES

`"OrigenID","DestinoID","Tiempo","Costo"`

PRODUCTOS

`"SucursalID","Nombre","CodigoBarra","Categoria","FechaCaducidad","Marca","Precio","Stock"`

Casos que generan un error en el log:

- Línea con menos o más de 7 campos
- Codigo de barra vacío
- Codigo de barra duplicado
- Precio no numérico o igual a cero
- Stock no entero o negativo
- Fecha con formato incorrecto (no YYYY-MM-DD)
- Mes fuera del rango 01-12 o dia fuera del rango 01-31
- Categoría o marca vacias

```
[2026-05-03 16:44:19] MALFORMED line 5: ID invalido:
[2026-05-03 16:44:19] MALFORMED line 6: TiempoIngreso invalido
[2026-05-03 16:44:19] MALFORMED line 7: Nombre o Ubicacion vacios
[2026-05-03 16:44:19] INFO: [CSVLoader] Sucursales: 3 cargadas, 3 omitidas.
[2026-05-03 16:44:20] ERROR: Sucursal origen no existe: 99
[2026-05-03 16:44:20] MALFORMED line 8: DestinoID invalido:
[2026-05-03 16:44:20] MALFORMED line 9: Tiempo invalido
[2026-05-03 16:44:20] INFO: [CSVLoader] Conexiones: 5 cargadas, 3 omitidas.
[2026-05-03 16:44:22] MALFORMED line 7: Campos texto vacios
[2026-05-03 16:44:22] MALFORMED line 8: Campos texto vacios
[2026-05-03 16:44:22] MALFORMED line 9: Precio invalido: precio_invalido
[2026-05-03 16:44:22] MALFORMED line 10: Campos texto vacios
[2026-05-03 16:44:22] MALFORMED line 11: FechaCaducidad invalida: fecha-mala
```

9. <u>Ejecutar benchmark:</u>

Tabla	Que mide	Estructuras comparadas
Búsqueda	Tiempo de búsqueda por nombre en 4 casos	LinkedList, SortedList, AVL
Inserción	Tiempo de inserción individual	LinkedList, SortedList, AVL, BTree, B+
Eliminación	Tiempo de eliminación individual	LinkedList, SortedList, AVL, BTree, B+

Los tiempos se reportan en microsegundos (us) con valor promedio, minimo y maximo

Sucursal: Sucursal Solola N=20 operaciones · M=5 repeticiones · tiempos promedio por operación individual

Búsqueda Inserción Eliminación

LinkedList · AVL buscan por **nombre**. HashTable busca por **barcode** (aleatorio y miss únicamente). B-Tree busca por **fecha de vencimiento**. B+ Tree busca por **categoria**.

Estructura	Aleatorio	Miss	Mejor caso	Peor caso
LinkedList	0.42 µs ↓0.29 µs ↑0.64 µs	0.57 µs ↓0.47 µs ↑0.70 µs	0.34 µs ↓0.34 µs ↑0.35 µs	0.45 µs ↓0.44 µs ↑0.47 µs
HashTable	0.92 µs ↓0.60 µs ↑2.03 µs	0.78 µs ↓0.57 µs ↑1.51 µs	—	—
AVL	1.01 µs ↓0.51 µs ↑2.74 µs	0.48 µs ↓0.33 µs ↑0.69 µs	0.36 µs ↓0.31 µs ↑0.45 µs	0.32 µs ↓0.29 µs ↑0.36 µs
B-Tree	5.90 µs ↓0.27 µs ↑28.19 µs	0.27 µs ↓0.21 µs ↑0.33 µs	0.26 µs ↓0.19 µs ↑0.33 µs	0.30 µs ↓0.22 µs ↑0.33 µs
B+ Tree	0.39 µs ↓0.34 µs ↑0.47 µs	0.34 µs ↓0.29 µs ↑0.38 µs	0.38 µs ↓0.27 µs ↑0.49 µs	0.47 µs ↓0.38 µs ↑0.53 µs

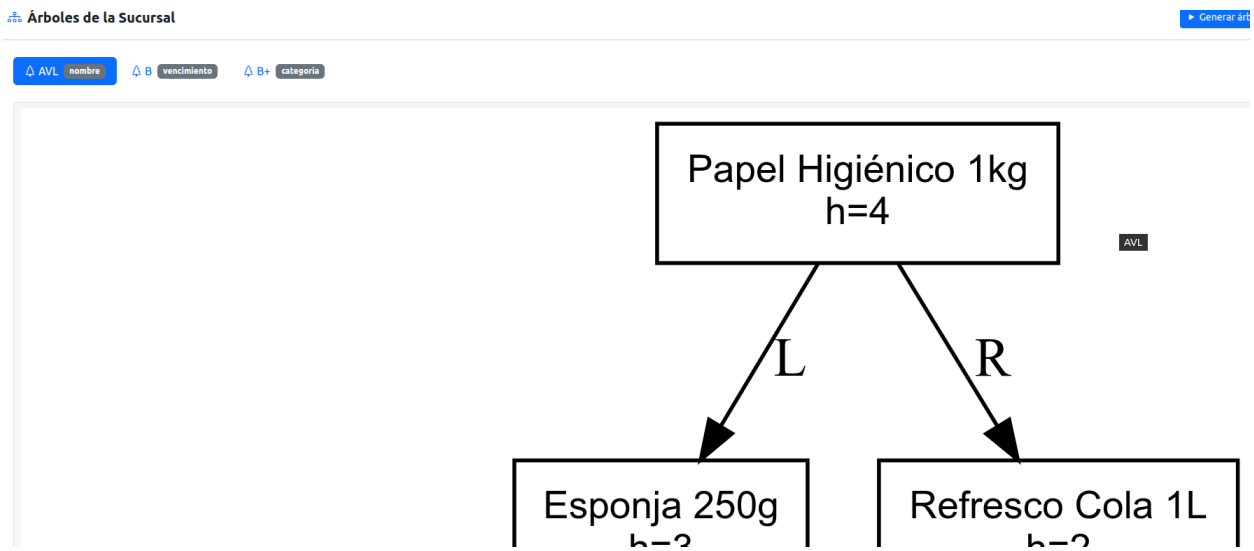
Tiempos promedio por operación individual. **Verde** < 10 µs **Amarillo** < 100 µs **Rojo** ≥ 100 µs ↓ min ↑ m

10. <u>Generar archivos .dot y PNG:</u> Solicita un nombre para los archivos (ej. prueba1). Genera seis archivos en la carpeta resultados/:

Archivo generado	Contenido
<nombre>_AVL.dot / .png	Árbol AVL ordenado por nombre
<nombre>_BTree.dot / .png	Árbol B ordenado por fecha de caducidad
<nombre>_BPlus.dot / .png	Arbol B+ con categorias en las hojas

La generación de PNG requiere que Graphviz esté instalado. Si no está disponible, los archivos .dot se generan igual y pueden convertirse manualmente con:

```
dot -Tpng resultados/prueba1_AVL.dot -o resultados/prueba1_AVL.png
```



Archivo de errores

El archivo resultados/errors.log registra todos los errores durante la carga de archivos CSV. Se crea automáticamente al iniciar el programa y se escribe en modo append para conservar el historial de sesiones anteriores.

Formato de cada entrada:

```
[YYYY-MM-DD HH:MM:SS] TIPO: descripción
```